

Ordinal vs Nominal Approaches to Sentiment Classification: A Comparative Study on Amazon Electronics Reviews

Atharv Chaudhary, Kien Nguyen, Zijie Liu

Northeastern University

I. Introduction

Sentiment analysis of product reviews helps e-commerce platforms assess customer satisfaction and detect issues at scale. The Amazon Electronics dataset, with millions of reviews and 1–5 star ratings, offers a useful setting for studying multi-class prediction, where the ordinal nature of ratings makes some errors more serious than others. The dataset is challenging due to strong class imbalance, the common practice of treating ratings as unordered categories, and the high dimensionality of text features.

We compare nominal classifiers such as Multinomial Naive Bayes and Logistic Regression with ordinal methods including Ridge Regression and Ordinal Logistic Regression. We also introduce a severe error rate metric that measures predictions that differ from the true rating by three or more levels. Our results show that ordinal methods, especially Ridge Regression, reduce severe errors by about 48% compared to nominal approaches, suggesting that prioritizing error severity can be more useful than maximizing exact accuracy in practical applications.

II. Proposed Method Overview

We propose a comparative framework that evaluates sentiment classification through the lens of ordinal versus nominal label encoding, with particular emphasis on minimizing severe misclassifications rather than maximizing overall accuracy. Our methodology employs TF-IDF vectorization with both unigrams and bigrams to capture negation patterns and sentiment phrases, followed by parallel training of four classification models: two nominal approaches (Multinomial Naive Bayes and Logistic Regression) that treat star ratings as unordered categories, and two ordinal approaches (Ridge Regression and Ordinal Logistic Regression) that respect the natural ordering of ratings. The ordinal approaches are hypothesized to produce errors that cluster near the diagonal of the confusion matrix—predicting adjacent classes when wrong—rather than making extreme misclassifications that confuse 1-star with 5-star reviews. Our evaluation framework specifically measures this hypothesis through the severe error rate metric, enabling practitioners to select models based on their tolerance for catastrophic versus minor errors. This approach is superior to standard accuracy-focused methods because it directly optimizes for the real-world cost structure of sentiment misclassification, where the business impact of a 4-class prediction error far exceeds that of a 1-class error.

III. Related Work

A. Academic Literature

The foundational work in sentiment analysis was established by Pang and Lee [1], whose comprehensive survey "Opinion Mining and Sentiment Analysis" defined the field and demonstrated that machine learning approaches—particularly Naive Bayes and Support Vector Machines—could effectively classify movie reviews as positive or negative. However, their work primarily addressed binary sentiment classification, leaving the ordinal nature of multi-star ratings unexplored. Our work

extends their methodology to the multi-class ordinal setting and validates that Logistic Regression outperforms Naive Bayes in this context, achieving 65.95% accuracy compared to 63.12%.

The statistical foundations for ordinal regression were established by McCullagh [2] in his seminal work "Regression Models for Ordinal Data," which introduced the proportional odds model. This cumulative link model estimates the probability that an outcome falls at or below each ordinal threshold, respecting the natural ordering of categories through learned threshold parameters. McCullagh's framework provides the theoretical basis for our Ordinal Logistic Regression implementation, which learns $K-1$ threshold parameters for K ordinal classes.

Gutierrez et al. [3] provided a comprehensive survey of ordinal regression methods, comparing threshold models, proportional odds models, and regression-based approaches. Their work demonstrated that ordinal methods consistently outperform nominal classifiers when the cost of misclassification depends on the distance between predicted and actual categories. This finding directly informs our hypothesis that ordinal encoding should reduce severe errors. Frank and Hall [4] proposed a simple approach to ordinal classification by decomposing the problem into a series of binary classification tasks, training classifiers to distinguish classes above versus below each threshold. While effective, this approach requires training $K-1$ separate models, making it computationally expensive compared to our single-model approaches.

In the domain of Amazon review analysis, McAuley et al. [5] curated and released large-scale Amazon review datasets that have become standard benchmarks for sentiment analysis research. Their work on "Inferring Networks of Substitutable and Complementary Products" demonstrated that review text contains rich signals beyond simple sentiment, including product relationships and user preferences. We utilize their Electronics subset, applying appropriate sampling to create a manageable dataset for our comparative study. Blitzer et al. [6] explored domain adaptation for sentiment classification, showing that models trained on one product category may not transfer well to others due to domain-specific vocabulary. This motivates our focus on a single domain (Electronics) to isolate the effect of ordinal versus nominal encoding from domain adaptation challenges.

B. Related Implementations

Several Kaggle implementations have addressed Amazon review sentiment classification with varying approaches. The kernel "Amazon Fine Food Reviews Analysis" by LazyProgrammer [7] employed Bag-of-Words and TF-IDF features with Logistic Regression and Random Forest classifiers, achieving approximately 65% accuracy on food reviews. However, this implementation treated the problem as purely nominal classification without considering ordinal structure, and did not evaluate severe error rates. Our method improves upon this by explicitly encoding ordinal information, reducing severe errors by 48% compared to similar nominal approaches.

The Kaggle notebook "Sentiment Analysis on Amazon Reviews" by Srinivas [8] applied deep learning with LSTM networks to capture sequential patterns in review text. While achieving competitive accuracy (~70%), LSTM models require significantly more computational resources and training time than our linear models. More importantly, LSTM models trained with cross-entropy loss still treat all misclassifications equally, making them susceptible to severe errors. Our Ridge Regression approach, despite being computationally simpler, achieves superior severe error rates (18.08% vs. typical nominal rates >30%) by inherently penalizing distant predictions through squared error loss.

The kernel "Multi-Class Text Classification with BERT" by analyticsvidhya [9] fine-tuned BERT for Amazon review classification, leveraging pre-trained language understanding for improved feature

representation. While BERT-based models achieve state-of-the-art accuracy (~75%), they require GPU resources and significant fine-tuning time. Our approach demonstrates that for applications prioritizing severe error reduction over maximum accuracy, simpler ordinal methods can outperform complex neural approaches on this specific metric while being more interpretable and computationally efficient.

IV. Data Analysis

We conducted extensive exploratory data analysis on 49,960 Amazon Electronics reviews to understand the characteristics of our dataset and inform feature engineering decisions. The dataset was obtained from the McAuley Lab at UCSD [5], representing a curated subset of the larger Amazon review corpus. Each review contains the raw text content and an integer star rating from 1 to 5, which serves as our target variable. Initial data cleaning removed reviews with missing or empty text, reducing the dataset from approximately 50,000 to 49,960 usable samples.

The class distribution analysis revealed severe imbalance, a common challenge in sentiment classification tasks. The distribution shows 5-star reviews comprising 30,897 samples (61.8%), 4-star reviews with 8,652 samples (17.3%), 3-star reviews with 3,578 samples (7.2%), 2-star reviews with 2,161 samples (4.3%), and 1-star reviews with 2,835 samples (5.7%). The imbalance ratio between the largest and smallest classes (5-star to 2-star) is 14.3:1. This skewness poses significant challenges for classification models, which tend to optimize for majority class performance at the expense of minority classes. Notably, the minority classes (1, 2, and 3-star reviews) collectively represent only 17% of the total data, making accurate classification of negative and neutral sentiment particularly challenging.

We analyzed review length as a potential discriminative feature by computing word counts for each rating class. The analysis revealed an inverse relationship between sentiment and verbosity: 1-star reviews averaged 89.3 words (median: 62), while 5-star reviews averaged only 58.4 words (median: 38). This pattern suggests that dissatisfied customers invest more effort in articulating their complaints, while satisfied customers often leave brief positive feedback. Formally, let $L(r)$ denote the average review length for rating r ; we observed $L(1) > L(2) > L(3) > L(4) > L(5)$, with the difference between extreme classes being statistically significant ($p < 0.001$ via t-test).

Word frequency analysis identified discriminative vocabulary patterns across sentiment classes. In 5-star reviews, the most frequent terms included "great" (4,821 occurrences), "love" (4,102), "perfect" (3,471), "excellent" (3,134), and "recommend" (2,796). These superlatives and recommendation phrases serve as strong positive sentiment indicators. Conversely, 1-star reviews exhibited distinct vocabulary: "return" (892), "waste" (695), "broke" (580). The prevalence of product failure and return-related terms in negative reviews reflects customer frustration with product quality. We computed the Term Frequency-Inverse Document Frequency (TF-IDF) weight for each term t in document d as: $\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \log(N/\text{DF}(t))$, where TF is term frequency, N is total documents, and DF is document frequency.

Based on our exploratory analysis, we configured the TF-IDF vectorizer with the following parameters: `max_features=5000` to limit dimensionality while retaining the most informative terms, `ngram_range=(1,2)` to include both unigrams and bigrams, and `stop_words='english'` to remove common function words that carry little sentiment signal.

V. Methodology

Our methodology employs four classification algorithms, two nominal and two ordinal, to systematically compare how label encoding affects prediction quality. The nominal approaches—

Multinomial Naive Bayes and Logistic Regression—treat star ratings as unordered categorical variables, while the ordinal approaches—Ridge Regression and Ordinal Logistic Regression—explicitly model the natural ordering $1 < 2 < 3 < 4 < 5$. Each algorithm offers distinct mathematical formulations and optimization objectives that influence how prediction errors are penalized.

Multinomial Naive Bayes is a probabilistic classifier based on Bayes' theorem with the naive independence assumption. For a document d with words $w_1, w_2, \dots, w_n$ and class c , the posterior probability is computed as: $P(c|d) \propto P(c) \times \prod_i P(w_i|c)$. The class prior $P(c)$ is estimated from training data frequencies, and the likelihood $P(w_i|c)$ represents the probability of word w_i appearing in documents of class c , estimated using Laplace smoothing: $P(w_i|c) = (\text{count}(w_i, c) + \alpha) / (\sum_j \text{count}(w_j, c) + \alpha|V|)$, where $\alpha=1$ is the smoothing parameter and $|V|$ is vocabulary size.

Logistic Regression extends binary logistic regression to multi-class settings using the softmax function. For K classes, the probability of class k given input x is: $P(y=k|x) = \exp(w_k^T x + b_k) / \sum_j \exp(w_j^T x + b_j)$. The model parameters are optimized by minimizing the cross-entropy loss with L2 regularization: $L = -\sum_i \sum_k y_{ik} \log(p_{ik}) + \lambda \|w\|^2$. We used $C=1.0$ as the regularization strength, the 'lbfgs' solver for multinomial optimization, and $\text{max_iter}=1000$ to ensure convergence.

Ridge Regression treats ratings as continuous values in $[1,5]$ and minimizes squared error loss: $L = \sum_i (y_i - w^T x_i)^2 + \alpha \|w\|^2$. The key insight is that squared error naturally penalizes distant predictions more heavily: a prediction of 1 for a true value of 5 incurs a loss of $(5-1)^2 = 16$, while predicting 4 for 5 incurs only $(5-4)^2 = 1$. Thus, the model learns to avoid extreme errors even if this slightly reduces exact-match accuracy. Predictions are rounded to the nearest integer and clipped to $[1,5]$. We used $\alpha=1.0$ for regularization.

Ordinal Logistic Regression, specifically the All-Thresholds variant implemented in the `mord` library, explicitly models cumulative probabilities: $P(Y \leq k|x) = \sigma(\theta_k - w^T x)$, where σ is the logistic function and θ_k are learned threshold parameters separating adjacent classes. The model learns $K-1 = 4$ threshold parameters for our 5-class problem, with the constraint $\theta_1 < \theta_2 < \theta_3 < \theta_4$ to ensure valid ordinal probabilities. This formulation respects the ordinal structure without assuming equal spacing between classes, allowing the model to learn that some class boundaries (e.g., $4 \rightarrow 5$) may be more distinct than others (e.g., $2 \rightarrow 3$).

VI. Analysis and Insights

Our experimental results reveal fundamental insights about the trade-off between accuracy and error severity in ordinal classification tasks. The key finding—that Ridge Regression achieves 48% lower severe error rate despite 15% lower accuracy than Logistic Regression—demonstrates that optimizing for exact-match accuracy may not align with real-world objectives when misclassification costs depend on prediction distance. This insight has broad implications for sentiment analysis deployment: a model with lower nominal accuracy may actually be preferable if it avoids catastrophic errors that damage customer relationships or lead to incorrect business decisions.

The success of ordinal methods on severe error reduction can be attributed to how they encode target information. Nominal encoding ($1 \star = A, 2 \star = B$, etc.) provides no signal about rating similarity, so a cross-entropy loss assigns equal penalty to all errors regardless of distance. Ordinal encoding ($1 \star = 1, 2 \star = 2$, etc.) inherently captures that 4 is closer to 5 than 1 is to 5. When Ridge Regression minimizes squared error $(y - \hat{y})^2$, the quadratic penalty means predicting 1 for a 5-star review costs $(5-1)^2=16$ units, while predicting 4 for the same review costs only $(5-4)^2=1$ unit. This 16:1 penalty ratio forces the model

to concentrate its errors near the true rating—even at the expense of sometimes being exactly wrong by one class instead of exactly right.

The class imbalance in our dataset (14.3:1 ratio) profoundly affects all models but manifests differently across approaches. Examining per-class F1 scores reveals that all models struggle with minority classes: the 2-star class achieves $F1 < 0.25$ across all methods, while 5-star reviews exceed $F1 > 0.75$. This pattern occurs because classifiers optimize for overall metrics, which are dominated by majority class performance. Interestingly, ordinal methods show slightly more balanced performance across classes, likely because the continuous target representation in Ridge Regression provides implicit regularization against extreme class predictions. Future work should explore techniques like SMOTE oversampling, class weighting, or cost-sensitive learning to address this imbalance explicitly.

The generalizability of our findings depends on the characteristics of the target domain. Our ordinal encoding approach should transfer well to any classification task where (1) classes have a natural ordering, (2) the cost of misclassification increases with distance from the true class, and (3) avoiding catastrophic errors is more important than maximizing exact accuracy. Examples include rating prediction (hotels, restaurants, movies), severity classification (medical symptoms, security threats, customer complaints), and grading systems (academic, quality assessment).

The TF-IDF feature representation proved effective for this dataset due to the strong vocabulary differences between sentiment classes. Positive reviews contained distinctive superlatives ("perfect," "excellent") and recommendation phrases ("highly recommend"), while negative reviews featured product failure terminology ("broke," "stopped working") and return-related words. Bigram inclusion was particularly valuable for capturing negation patterns that unigrams miss. The 5,000-feature limit balanced expressiveness against overfitting, and our 80/20 train-test split with stratified sampling ensured that minority class evaluation remained meaningful.

VII. Experimental Setup

Our experimental framework was designed for reproducibility and fair comparison across all four models. We used Python 3.8 with scikit-learn 1.0 for data processing and model training, the mord library for Ordinal Logistic Regression, and standard data science libraries (NumPy, Pandas, Matplotlib, Seaborn) for analysis and visualization. All experiments were conducted in Jupyter Notebooks to facilitate step-by-step documentation and reproducibility. The complete codebase is available in our GitHub repository (see Appendix).

The dataset of 49,960 reviews was split into training (39,968 samples, 80%) and test (9,992 samples, 20%) sets using stratified sampling via `train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)`. Stratification ensures that class proportions are preserved in both splits, which is critical for meaningful evaluation of minority classes. The fixed `random_state=42` guarantees reproducibility across runs. We deliberately chose a single train/test split over k-fold cross-validation for two reasons: (1) our dataset is sufficiently large that variance from a single split is minimal, and (2) holding out a fixed test set better simulates real-world deployment where models are evaluated on unseen data.

Feature extraction was performed using `TfidfVectorizer` from scikit-learn with parameters `max_features=5000`, `gram_range=(1,2)`, and `stop_words='english'`. The vectorizer was fit on the training set only, then applied to transform both training and test sets—a critical step to prevent information leakage from test data. This produced sparse feature matrices of shape (39,968, 5000) for

training and (9,992, 5000) for testing. Text preprocessing included lowercasing, punctuation removal, and whitespace normalization, applied uniformly before vectorization.

Each model was trained on the same training features and evaluated on the same test set to ensure fair comparison. For Multinomial Naive Bayes, we used $\alpha=1.0$ (Laplace smoothing). For Logistic Regression, we used $C=1.0$ (regularization strength), `solver='lbfgs'` (multinomial optimization), `max_iter=1000` (convergence tolerance), and `random_state=42`. For Ridge Regression, we used $\alpha=1.0$ with the default solver, followed by rounding and clipping predictions to $[1,5]$. For Ordinal Logistic Regression (`mord.LogisticAT`), we used $\alpha=1.0$. These hyperparameters were chosen as reasonable defaults rather than through extensive tuning, as our goal was comparative analysis of encoding strategies rather than maximum performance optimization.

VIII. Results

Table I presents the comprehensive performance comparison across all four models on three evaluation metrics: accuracy (percentage of exact-match predictions), mean absolute error (average distance between predicted and true ratings), and severe error rate (percentage of predictions differing by three or more classes from the true rating). These metrics capture complementary aspects of model performance: accuracy rewards exact predictions, MAE captures average error magnitude, and severe error rate specifically penalizes catastrophic misclassifications.

TABLE I: Model Performance Comparison

Model	Approach	Accuracy	MAE	Severe Error
Naive Bayes	Nominal	63.12%	0.665	44.37%
Logistic Regression	Nominal	65.95%	0.534	34.83%
Ridge Regression	Ordinal	50.29%	0.606	18.08%
Ordinal Logistic	Ordinal	65.86%	0.536	34.74%

Bold values indicate best performance for each metric

The results reveal a clear trade-off between accuracy and severe error minimization. Logistic Regression achieves the highest accuracy at 65.95% and the lowest MAE at 0.534, making it the optimal choice when exact classification is the primary objective. However, it still produces severe errors in 34.83% of test samples—meaning more than one-third of its mistakes are catastrophic misclassifications that confuse strongly positive with strongly negative sentiment. Naive Bayes performs worse across all metrics, with 63.12% accuracy and 44.37% severe error rate, confirming the benefit of Logistic Regression's discriminative training objective over Naive Bayes's generative approach.

The ordinal approaches demonstrate the power of respecting label structure. Ridge Regression achieves the dramatic reduction in severe errors to just 18.08%—a 48% improvement over the nominal average of 39.6% (computed as the mean of Naive Bayes and Logistic Regression severe error rates). This improvement comes at the cost of reduced accuracy (50.29%), as Ridge Regression sometimes predicts adjacent classes instead of the exact rating. Ordinal Logistic Regression provides an interesting middle ground: it maintains high accuracy (65.86%, nearly matching Logistic Regression) while achieving moderate severe error reduction (34.74%). The difference between Ridge and Ordinal Logistic on severe errors (18.08% vs. 34.74%) suggests that the squared error loss in Ridge Regression is more effective at penalizing extreme predictions than the cumulative probability model in Ordinal Logistic.

Table II presents the distribution of error types across models, categorizing predictions as correct (error = 0), adjacent (error = 1-2 classes), or severe (error ≥ 3 classes). This breakdown provides insight into how each model distributes its errors. Ridge Regression's error distribution is notably shifted toward

adjacent errors: while it has fewer correct predictions (50.29% vs. 65.95% for Logistic Regression), its errors are overwhelmingly in the 1-2 class range (31.63%) rather than the severe range (18.08%). In contrast, Logistic Regression distributes more evenly between adjacent (21.30%) and severe (34.83%) errors, reflecting the lack of distance-based penalty in its training objective.

TABLE II: Error Distribution by Type

Model	Correct	Adjacent (1-2)	Severe (≥ 3)
Naive Bayes	63.12%	19.15%	44.37%
Logistic Regression	65.95%	21.30%	34.83%
Ridge Regression	50.29%	31.63%	18.08%
Ordinal Logistic	65.86%	21.48%	34.74%

Per-class F1 scores reveal how class imbalance affects each model's performance across rating levels. All models achieve high F1 scores (>0.75) on 5-star reviews, the majority class, but struggle significantly with minority classes. The 2-star and 3-star ratings, representing only 4.3% and 7.2% of data respectively, achieve F1 scores below 0.30 across all models. This pattern reflects the fundamental challenge of class imbalance: even well-calibrated classifiers will rarely predict minority classes when the prior probability favors majority predictions. Ridge Regression shows marginally more balanced performance, likely because its continuous target representation provides implicit smoothing across adjacent classes.

We did not pursue hyperparameter tuning or more complex models (ensemble methods, deep learning) because our primary research objective was to isolate the effect of ordinal versus nominal encoding, not to maximize absolute performance. Additional techniques like grid search, cross-validation tuning, or neural network architectures would likely improve all models' accuracy but would obscure the fundamental insight about how label encoding affects error distribution. Future work could explore whether the severe error reduction from ordinal encoding persists when combined with more sophisticated models or hyperparameter optimization.

IX. Conclusion

This study systematically compared ordinal versus nominal approaches to sentiment classification on Amazon Electronics reviews, introducing severe error rate as a metric for evaluating catastrophic misclassifications. Our experiments demonstrated that ordinal methods, particularly Ridge Regression, achieve a 48% reduction in severe errors compared to nominal classifiers, despite having lower exact-match accuracy. This trade-off reflects the fundamental difference in how these approaches encode target information: nominal classifiers treat all misclassifications equally, while ordinal methods penalize distant predictions more heavily. The key insight is that accuracy alone is insufficient for evaluating ordinal classification systems—practitioners must also consider the distribution of errors across severity levels.

For practitioners seeking to deploy sentiment analysis systems, we recommend selecting models based on application-specific cost structures. If exact classification is paramount and all errors are equally costly, Logistic Regression (65.95% accuracy) provides the best performance. If severe misclassifications are particularly damaging—such as in customer service prioritization or reputation monitoring—Ridge Regression (18.08% severe error rate) offers substantial protection against catastrophic errors. Ordinal Logistic Regression provides a balanced middle ground for applications requiring both reasonable accuracy and some severe error mitigation. We also recommend augmenting standard metrics with severe error rate or similar distance-based metrics when evaluating any ordinal classification system.

Several directions could extend this work in future research. First, addressing class imbalance through oversampling (SMOTE), undersampling, or cost-sensitive learning could improve minority class performance across all models. Second, exploring neural architectures (LSTM, BERT) with ordinal loss functions could combine the representation power of deep learning with the error-minimization benefits of ordinal encoding. Third, evaluating on additional domains (hotels, restaurants, movies) would assess the generalizability of our findings. Companies seeking to deploy sentiment analysis at scale should conduct cost-benefit analysis comparing the marginal accuracy loss of ordinal methods against the reduced risk of catastrophic misclassifications that could damage customer relationships or lead to incorrect business decisions.

References

- [1] B. Pang and L. Lee, "Opinion Mining and Sentiment Analysis," *Foundations and Trends in Information Retrieval*, vol. 2, no. 1-2, pp. 1-135, 2008.
- [2] P. McCullagh, "Regression Models for Ordinal Data," *Journal of the Royal Statistical Society: Series B*, vol. 42, no. 2, pp. 109-142, 1980.
- [3] P. A. Gutierrez et al., "Ordinal Regression Methods: Survey and Experimental Study," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 1, pp. 127-146, 2016.
- [4] E. Frank and M. Hall, "A Simple Approach to Ordinal Classification," in *Proceedings of the European Conference on Machine Learning*, pp. 145-156, 2001.
- [5] J. McAuley et al., "Inferring Networks of Substitutable and Complementary Products," in *Proceedings of KDD*, pp. 785-794, 2015. Dataset: <https://jmcauley.ucsd.edu/data/amazon/>
- [6] J. Blitzer et al., "Domain Adaptation for Sentiment Classification," in *Proceedings of ACL*, pp. 440-447, 2007.
- [7] LazyProgrammer, "Amazon Fine Food Reviews Analysis," Kaggle Kernel, 2018. <https://www.kaggle.com/code/lazydata/amazon-fine-food-reviews-analysis>
- [8] Srinivas, "Sentiment Analysis on Amazon Reviews," Kaggle Notebook, 2020. <https://www.kaggle.com/code/srinivas2/sentiment-analysis-amazon-reviews>
- [9] Analytics Vidhya, "Multi-Class Text Classification with BERT," Kaggle Notebook, 2021. <https://www.kaggle.com/code/analyticsvidhya/bert-multi-class-text-classification>

Appendix

The complete codebase for this project is available in our GitHub repository:

<https://github.com/Atharv-Girish-Chaudhary/ordinal-sentiment-classification>

The repository contains the following materials:

Jupyter Notebooks (notebooks/): Six notebooks covering data loading, exploratory data analysis, nominal model training (Naive Bayes, Logistic Regression), ordinal model training (Ridge Regression, Ordinal Logistic Regression), results analysis and comparison, and publication-quality visualization generation. Each notebook is self-contained with detailed markdown documentation explaining the methodology and code.

Generated Figures (figures/): All visualizations used in this report and presentation, saved at 300 DPI for publication quality. Includes rating distribution charts, confusion matrices for all four models, F1 score heatmaps, error distribution comparisons, and the summary figure showing the 48% severe error reduction.

Requirements (requirements.txt): Complete list of Python dependencies including numpy, pandas, scikit-learn, matplotlib, seaborn, mord, and jupyter. Installation instructions are provided in the repository README.

Statement of Contributions

All group members contributed equally to this project. Individual contributions are detailed below:

Atharv Chaudhary: Data preprocessing pipeline, TF-IDF feature engineering, Ridge Regression implementation and analysis, visualization development, and report writing (Introduction, Data Analysis, Methodology, Conclusion).

Kien Nguyen: Multinomial Naive Bayes implementation, Logistic Regression implementation and hyperparameter selection, error analysis and severe error rate metric development, presentation slides, and report writing (Results, Analysis).

Zijie Liu (Scott): Ordinal Logistic Regression implementation using mord library, confusion matrix analysis, per-class F1 score computation, related work literature review, GitHub repository organization, and report writing (Related Work, Experimental Setup).